Sealed: Cryptographic Supply Chain Attestation for Python Packages

Tushar Sharma
ALIA Labs
txshar@proton.me
https://github.com/TxsharDev/sealed

June 2026

Abstract

Software supply chain attacks exploit a fundamental gap: developers install pre-built binaries without verifying they match the published source code. Existing tools address pieces of this problem – Sigstore for signing, in-toto for multi-party layouts, SLSA as a framework – but none provide a single-command, zero-configuration solution for the Python ecosystem. We present Sealed, a tool that builds Python packages from source, records a five-step provenance chain (environment attestation, source verification, toolchain capture, build output), signs the chain with Ed25519, and verifies it before installation. Sealed adds recursive transitive dependency sealing, trust-on-first-use key pinning, key revocation, multi-party verification, a shared seal registry, and a configurable trust policy engine. Cryptographic operations add under 0.2 ms per package. We validate the system with 325 tests covering tamper detection, signature forgery, key compromise, and policy enforcement. The entire flow is two commands: `pip install alia-sealed && sealed install <package>`.

1 Introduction

The software supply chain is a chain of trust decisions. When a developer runs `pip install requests`, they trust that:

1. PyPI served the correct package (not a typosquat or compromised mirror)
2. The binary matches the published source code
3. Nobody modified the binary between the build server and their machine
4. The build environment itself was not compromised

None of these assumptions are verified by default. The consequences are real: SolarWinds (2020) injected malware into the build process, affecting 18,000 organizations [1]. The xz utils backdoor (2024) was planted in build scripts by a trusted maintainer [2]. PyPI typosquatting attacks upload hundreds of malicious packages weekly [3].

Each of these attacks exploits a different vector. No single tool prevents all of them. But one specific gap is addressable today: verifying that the binary you install was built from the source you can inspect, on a known environment, with a cryptographic proof chain.

Existing solutions require infrastructure (Sigstore needs Rekor and Fulcio), coordination (in-toto needs layout files and multiple signers), or ecosystem replacement (Nix, Guix). For a Python developer who wants supply chain verification today, there is no single-command option.

Sealed fills this gap. Two commands:

```
pip install alia-sealed
sealed install requests
```

This builds `requests` and all its transitive dependencies from source, records provenance chains, signs them, checks the trust policy, and installs verified artifacts.

2 System Design

Sealed is a 22-module Python library with a CLI entry point. The architecture follows a pipeline: attest the environment, fetch source, build, record provenance, sign, verify, check policy, store in registry.

2.1 Provenance Chain

The core data structure is a **ProvenanceChain**: an ordered list of **ProvenanceRecords**, each recording an input hash, output hash, timestamp, and metadata. The chain includes a **BuildEnvironment** fingerprint (Python version, platform, architecture).

The chain hash is computed as:

$$H_{\text{chain}} = \text{SHA-256}(\text{pkg\_id} \| \text{env\_canonical} \| R_1 \| R_2 \| \cdots \| R_n) \tag{1}$$

where R_i is the canonical JSON serialization of record i (sorted keys, no whitespace), and env_canonical is the deterministic serialization of the build environment. The environment is included in the hash to prevent metadata-swap attacks where an attacker replaces the environment field without breaking the signature.

2.2 Five-Step Chain

Every sealed package produces a chain with five records:

1. **Environment Attestation.** Measures the build machine: Python binary hash, pip version, OS kernel, CPU architecture, compiler versions, and build-affecting environment variables (CC, CFLAGS, LDFLAGS, etc.). When TPM 2.0 hardware is available, extends a PCR with the software measurements and obtains a hardware quote.

2. **Source Verification.** Downloads the source distribution (sdist) from PyPI, verifies the SHA-256 hash against PyPI's registry (fail-closed: no hash means no download), extracts with path traversal protection, and records archive hash to directory hash.

3. **Toolchain Capture.** Hashes the Python interpreter binary. Records the exact compiler that will build the artifact.

4. **Build.** Runs `pip wheel --no-deps --no-binary :all:` on the source directory. Records source directory hash to artifact hash.

5. **Behavioral Sandbox.** Imports the package in an isolated subprocess with monkey-patched network, subprocess, file I/O, and environment variable access. Records any suspicious behaviors (network connections, subprocess spawning, sensitive file reads, secret access).

2.3 Signing

The entire chain hash is signed with Ed25519 (via PyNaCl/libsodium). The signature covers:

$$\sigma = \text{Ed25519-Sign}(sk, H_{\text{chain}}) \tag{2}$$

A **Seal** object stores: chain hash, signature, public key, timestamp, package name, and version. Verification checks the signature, then compares the chain hash against a fresh recomputation from the records.

2.4 Trust Policy Engine

All trust decisions flow through a configurable policy engine that evaluates five checks in order:

1. **Signature verification** – Ed25519 over chain hash
2. **Key pin check** – TOFU: first use pins the key, mismatch blocks
3. **Revocation check** – revoked keys are rejected
4. **Attestation level** – method must be in the required set
5. **Multi-party check** – N-of-M unique signers required

All checks must pass. Any failure rejects the package.

2.5 Shared Registry

Seals and chains are stored in a SQLite database at `~/.sealed/registry.db`. The registry supports store/lookup by package, version, or signing key; export/import for team sharing (JSON); key pin distribution; and key revocation with reason tracking. Teams export seals and distribute the file – teammates import and verify against shared seals without a central server.

3 Security Analysis

3.1 Threat Model

Sealed protects against threats where the attacker modifies the binary or chain after the source is published. It does not protect against malicious source code or a fully compromised build machine (without TPM attestation).

3.2 Comparison with Existing Tools

Sealed is not a replacement for these tools. Sigstore provides keyless signing with transparency logs. in-toto provides formal supply chain layout verification. `pip --require-hashes` pins artifact hashes but does not verify provenance or sign anything. `poetry lock` pins versions and hashes but trusts pre-built wheels from PyPI. Sealed occupies a different point in the design space: zero-config, local-first, build-from-source, with a full cryptographic proof chain.

3.3 What Sealed Does Not Claim

- Sealed does not make builds reproducible. Two machines building the same package produce different chain hashes.
- Sealed does not audit source code. A backdoor in the source will be faithfully built and sealed.

Table 1: Threat model summary.

Threat	Detected?	Mechanism
PyPI mirror tampering	Yes	SHA-256 fail-closed
Download MITM	Yes	Hash verification
Post-build binary modification	Yes	Artifact hash in chain
Cross-package seal replay	Yes	Package ID in chain hash
Version replay attack	Yes	Version in chain hash
Key compromise (detection)	Yes	TOFU pin mismatch
Single point of trust	Yes	Multi-party N-of-M
Build env change (detection)	Yes	Attestation in chain
Malicious import behavior	Yes	Behavioral sandbox
Malicious source code	No	Out of scope
Compromised build (no TPM)	No	Requires TPM attestation

Table 2: Feature comparison with existing supply chain tools.

Feature	Sealed	Sigstore	in-toto	pip hashes	poetry lock
Single-command UX	Yes	No	No	No	No
Zero config	Yes	No	No	Manual	Auto
Build from source	Yes	No	No	No	No
Env attestation	Yes	No	No	No	No
TOFU key pinning	Yes	N/A	No	No	No
Multi-party sigs	Yes	No	Yes	No	No
Behavioral sandbox	Yes	No	No	No	No
Offline operation	Yes	No	Yes	Yes	Yes
Transitive deps	Yes	N/A	Yes	Manual	Yes

- Without TPM attestation, a compromised build machine controls the key, the build, and the chain.
- The signing key is stored as plaintext on disk. HSM integration is future work.

4 Performance

We benchmarked every core operation on a desktop machine (AMD 9800X3D, 64 GB DDR5, NVMe SSD, Windows 11, Python 3.12.9). All timings are wall-clock means over multiple iterations.

4.1 Cryptographic Operations

The signing and verification overhead is negligible. Table 3 shows that the entire seal-then-verify cycle costs under 0.2 ms per package.

Signing takes 42 μs, verification 76 μs. The full verification path – JSON deserialization, chain hash recomputation, Ed25519 verify, trusted key check – costs 109 μs. Policy evaluation including TOFU pin lookup, revocation check, and multi-party check via SQLite costs 87 μs. These numbers are dominated by Python interpreter overhead, not cryptography.

Table 3: Cryptographic operation timings (Ed25519 via PyNaCl/libsodium).

Operation	Mean	Stdev	Iterations
Key generation	0.024 ms	0.070 ms	100
Seal signing	0.042 ms	0.011 ms	500
Signature verification	0.076 ms	0.017 ms	500
Full verify (JSON round-trip)	0.109 ms	0.015 ms	200
Policy evaluation (5 checks)	0.087 ms	0.009 ms	200

4.2 Directory Hashing

Sealed's `_hash_directory` function hashes every file in a directory tree for provenance records. We compared it against a raw hashlib implementation using the same algorithm.

Table 4: Directory hashing: Sealed vs raw hashlib (100 files, 4 KB each, 400 KB total).

Method	Mean (ms)	Stdev (ms)
Sealed `_hash_directory`	10.69	0.49
Raw hashlib (same algorithm)	10.68	0.43

Overhead is 0.1% – effectively zero. Sealed's hashing adds no abstraction cost over direct hashlib use.

4.3 Scale

Table 5 shows how hashing and chain operations scale with input size.

Table 5: Scaling behavior across file counts and chain lengths.

Operation	Size	Mean (ms)	Total data
Directory hash	10 files	1.21	40 KB
Directory hash	100 files	10.68	400 KB
Directory hash	1000 files	108.17	4 MB
Chain hash	4 records	0.022	–
Chain hash	20 records	0.095	–
Chain hash	100 records	0.450	–

Directory hashing scales linearly with file count (1.21 ms for 10 files, 108 ms for 1000 files). Chain hashing scales linearly with record count. A typical sealed package has 4 records, costing 22 μs for the chain hash.

4.4 Lockfile Operations

Lockfile operations are sub-millisecond even at 100 packages. The integrity check (re-hash and compare) costs 0.73 ms for 100 packages – negligible compared to network and build time.

4.5 Attestation

Software attestation takes 233 ms, dominated by the subprocess call to `pip --version`. The actual hashing of Python binary, version strings, and environment variables is sub-millisecond. This is a one-time cost per build session, not per package.

Table 6: Lockfile operation timings.

Operation	10 packages	100 packages
Generate	0.028 ms	0.287 ms
Serialize to JSON	0.061 ms	0.527 ms
Integrity check	0.084 ms	0.730 ms
Single package lookup	0.001 ms	

4.6 Where Time Actually Goes

The bottleneck in `sealed install` is never cryptography. For a typical package:

- Network: downloading sdist from PyPI – seconds
- Build: `pip wheel` from source – seconds to minutes (C extensions)
- Sealed overhead: attestation (233 ms, once) + hash + sign + verify + policy < 15 ms per package

Sealed's cryptographic overhead is less than 0.1% of total install time for any real package.

5 Related Work

Sigstore [4] provides keyless signing using OIDC identity and a transparency log (Rekor). It removes the need for key management but requires internet access and an identity provider. Sealed operates offline with local keys.

in-toto [5] verifies that a software supply chain followed a defined layout, with multiple parties signing off on each step. It provides stronger multi-party guarantees but requires layout specification files and organizational coordination.

SLSA [6] (Supply-chain Levels for Software Artifacts) is a graduated framework defining four levels of supply chain security. Sealed implements properties consistent with SLSA Level 2 (signed provenance) in a single command.

TUF [7] (The Update Framework) secures software update systems with role-based signing and snapshot/timestamp metadata. TUF secures distribution; Sealed secures the build.

Reproducible Builds [8] aims for bit-for-bit identical builds across machines. Sealed records provenance per-build rather than verifying cross-machine reproducibility. The approaches are complementary.

pip –require-hashes pins artifact hashes in requirements files. It verifies that the downloaded wheel matches a known hash, but does not verify provenance, does not sign anything, and requires manual hash management.

poetry lock generates a lockfile with pinned versions and hashes. Like pip hashes, it trusts pre-built wheels from PyPI and does not verify that the wheel was built from the published source.

6 Limitations and Future Work

Build time. Pure Python packages build in seconds. C-extension packages (numpy, scipy) require minutes and a compiler toolchain. This is inherent to building from source.

Not bit-reproducible. Different machines produce different chain hashes due to timestamps, environment differences, and non-deterministic build tools.

Key storage. The signing key is plaintext on disk. Future work includes HSM integration and OS keychain support (the `os_keychain` module is stubbed).

Python only. v0.1 targets pip/PyPI. The core chain, seal, registry, and policy modules are ecosystem-agnostic – extending to npm or cargo requires adapting the source fetcher, builder, and resolver.

Transparency log. A centralized, append-only log would detect equivocation (signing two different binaries for the same package-version). Planned as future work.

7 Conclusion

Sealed provides zero-configuration supply chain attestation for Python packages. It builds from source, records five-step provenance chains, signs them with Ed25519, resolves and seals transitive dependencies, pins signing keys via TOFU, supports multi-party verification, and stores seals in a shared registry with a configurable trust policy engine. Cryptographic operations add under 0.2 ms per package – less than 0.1% of real install time. The system is validated by 325 tests. It does not replace Sigstore, in-toto, or SLSA. It makes supply chain verification accessible to any Python developer who can run `pip install`.

References

[1] Peisert, S., Schneier, B., Okhravi, H., et al. *Perspectives on the SolarWinds Incident.* IEEE Security & Privacy, 19(2), 2021.

[2] Freund, A. *Backdoor in upstream xz/liblzma leading to SSH server compromise.* oss-security mailing list, March 2024.

[3] Vu, D.L., Pham, I., Liang, Y., et al. *Typosquatting and Combosquatting Attacks on the Python Ecosystem.* IEEE European Symposium on Security and Privacy, 2023.

[4] Newman, Z., Engler, J., Torres-Arias, S. *Sigstore: Software Signing for Everybody.* IEEE Symposium on Security and Privacy, 2022.

[5] Torres-Arias, S., Afzali, H., Kuppusamy, T.K., et al. *in-toto: Providing farm-to-table guarantees for bits and bytes.* USENIX Security Symposium, 2019.

[6] Google. *SLSA: Supply-chain Levels for Software Artifacts.* https://slsa.dev, 2021.

[7] Samuel, J., Mathewson, N., Cappos, J., Dingledine, R. *Survivable Key Compromise in Software Update Systems.* ACM CCS, 2010.

[8] Lamb, C., Zacchiroli, S. *Reproducible Builds: Increasing the Integrity of Software Supply Chains.* IEEE Software, 39(2), 2022.

[9] Dolstra, E., de Jonge, M., Visser, E. *Nix: A Safe and Policy-Free System for Software Deployment.* LISA, 2004.

[10] Bernstein, D.J., Duif, N., Lange, T., Schwabe, P., Yang, B.Y. *High-speed high-security signatures.* Journal of Cryptographic Engineering, 2(2), 2012.

[11] Python Software Foundation. *Python Package Index.* https://pypi.org, 2026.